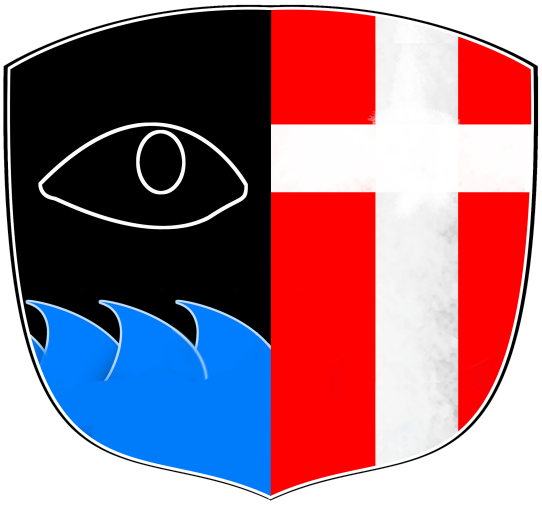
Full-Stack Platform for AIS Data Preprocessing and Visualization for Maritime Tanker Analysis



Sebastian Rix Neha Sharma

GitHub — Web App

June 1, 2026

Abstract

The project focuses on building a system for preprocessing, storing, and visualizing AIS tanker vessel data collected from large-scale maritime datasets. AIS data contains vessel information such as position, speed, direction, and navigational details, but raw AIS archives are difficult to process because they often contain duplicate records, missing values, invalid coordinates, inconsistent timestamps, and incomplete vessel identifiers.

To solve these problems, the system automatically collects and processes AIS tanker data using Python scripts, PostgreSQL databases, and Apache Airflow for task automation. Raw AIS records are first stored in a staging table, where the data is checked for missing values, duplicate records, invalid coordinates, and other inconsistencies before being cleaned and organized into structured database tables. The processed data is then made available through ASP.NET Core REST APIs and visualized through a web interface built with React and MapLibre.

The system supports tanker movement storage, anomaly detection, vessel tracking, and maritime monitoring. Overall, the project improves the accessibility, open-sourcing, and usability of AIS tanker data for future maritime analytics applications.

Contents

Abbreviations and glossary

AIS	Automatic Identification System
API	Application Programming Interface
COG	Course Over Ground
CSV	Comma-Separated Values
ETL	Extract, Transform, Load
ETA	Estimated Time of Arrival
GPS	Global Positioning System
IMO	International Maritime Organization
MMSI	Maritime Mobile Service Identity
REST	Representational State Transfer
ROT	Rate Of Turn
SOG	Speed Over Ground
SQL	Structured Query Language
UTC	Coordinated Universal Time
Idempotent	a data extraction or processing operation that yields the same output and has
Heterogeneous	a data sources that are a combination of datasets from different origins that va
Resource contention	A undesired scenario where multiple processes, applications, or tasks attempt t
URI versioning	URI versioning involves including the version number directly in the API endp
ORM	Object-Relational Mapping

1 Introduction

1.1 Background and Motivation

Maritime transport carries the great majority of global trade, and within it the movement of crude oil and petroleum products by tanker carries particular economic, strategic, and environmental weight. The principal mechanism for making this traffic transparent is the Automatic Identification System (AIS): vessels continuously broadcast their identity, position, course, and speed, and these messages are received by coastal stations and satellites to form a near-continuous record of maritime activity [1]. AIS has accordingly become a foundational data source for traffic monitoring, navigational safety, route analysis, and anomaly detection [2].

However, the same openness that makes AIS valuable also makes it manipulable. Because the identity and position fields are self-reported, they can be falsified: a vessel may transmit positions inconsistent with its true location or broadcast a false identity. Vessels can also carry a Maritime Mobile Service Identity (MMSI) whose national prefix resolves to no assigned maritime authority (country-code). Such identity and flag-irregularities are documented footprints of deceptive shipping practice. This is particularly true in sanctions-evasion contexts and the so-called shadow fleet of tankers [3]. Tankers are therefore precisely the vessel class in which deceptive AIS behavior is most consequential, which motivates this project's deliberate focus on them. Our goal is to ensuring that the tanker data is cleaned accessible at a large scale.

Surfacing this behavior from AIS data is primarily an infrastructure problem, not a modeling one. National AIS archives accumulate at a scale of billions of position reports, and the raw data is heterogeneous, has errors, duplicate records, malformed timestamps, invalid coordinates, and missing or inconsistent identifiers [4], [5]. Before any irregularity can be looked for, this volume must be ingested, cleaned, validated, and stored in a form that remains queryable at scale; and to be useful beyond a single analyst, the result must be made accessible and inspectable. The motivation for this project is therefore in two parts: to build a automated, scalable infrastructure that renders a complete historical tanker-AIS archive usable, and, within that infrastructure, to surface the identity- and flag-irregularity signals that are the indicators of deceptive behaviours. But also remaining honest about how far such signals can be interpreted as evidence. Lastly everything is hosted within EU , on our own physical hardware: Ubuntu-server, hard-discs and Router, to avoid foreign interference in global political environment shifting, and an icreassed focus on European IT sovereignty.

1.2 Problem Statement

The maritime domain continuously generates large-scale AIS data, and managing it manually is infeasible; effective use demands automated ingestion, cleaning, and storage. Raw AIS additionally contains defect data, duplication, invalid coordinates, malformed timestamps, and incomplete identifiers that must be resolved before analysis [4]. This is the first part of the problem: a scalable, automated pipeline is needed to turn a multi-billion-row year spanning AIS archive into clean, queryable, accessible data.

A second part of the problem concerns what such a pipeline should reveal. Because AIS identity and flag information is self-reported, it is susceptible to manipulation, and the resulting irregularities unresolved MMSI national prefixes, single hulls broadcasting multiple identities are recognized indicators of deceptive shipping practice [3]. Yet much existing work concentrates on analytical and machine-learning models applied to comparatively small, curated datasets, with less attention paid to the infrastructure required to apply even straightforward irregularity screening across a *complete* historical archive. Performing this at full scale introduces its own difficulty: aggregations spanning the entire corpus are computationally expensive, and naive whole-history approaches can prove infeasible on modest hardware.

There is therefore a need for a system that (i) ingests, cleans, and stores large-scale tanker AIS data through an automated, reproducible workflow; (ii) surfaces identity- and flag-irregularity signals within that workflow in a manner that remains tractable at archive scale; and (iii) makes both the processed data and the surfaced signals accessible and inspectable. A final, frequently neglected requirement is interpretive honesty: because confirming deceptive behavior would require labeled ground truth that such archives do not provide. The system must acknowledge *surfacing candidate signals* does not necessarily correlate with *illicit activity*. This means that just because our system nominates a vessel, does not mean it is participating in an illegal activity, which such determinations would probably be out of reach of this project.

2 Research Objective and Questions

2.1 Research Objective

The objective of this project is to design and implement an automated, scalable platform that ingests new and historical AIS record of tanker vessels and renders it both accessible and analytically useful and, within that platform, to develop screening that surfaces signals of vessel identity and flag irregularity at archive scale. The specific objectives are:

1. To ingest, clean, validate, and store large-scale tanker AIS data through an automated and reproducible workflow.
2. To orchestrate ingestion with prioritized, automated scheduling so that routine, on-demand, and backfill workloads coexist without resource contention or accidental data pollution.
3. To surface vessel identity and flag-irregularity signals through idempotent detection integrated with the pipeline.
4. To expose the processed data and the surfaced signals through a versioned REST API.
5. To provide an interactive web-based visualization through which vessels and flagged irregularities can be inspected.
6. To characterize the computational limits of archive-scale detection.

2.2 Research Questions

The overarching research question is:

> *How can large-scale tanker AIS data be transformed into an accessible analytical platform that surfaces signals of vessel identity and flag irregularity at archive scale, and what are the computational and evidentiary limits of doing so?*

This is decomposed into the following sub-questions:

1. How can a multi-billion-row AIS archive be ingested, cleaned, validated, and stored through an automated, reproducible workflow that remains computational?
2. How can ingestion be scheduled and prioritized so that routine, on-demand, and backfill workloads coexist without resource contention?

3. How can vessel identity and flag-irregularity signals such as unresolved MMSI national prefixes and inconsistencies between a hull's IMO and its broadcast MMSI be detected within such a pipeline?

4. What are the computational limits of applying such detection across a complete historical archive, and how do they shape the system architecture?

5. How can the processed data and the surfaced signals be made accessible and inspectable through a REST API and an interactive visualization?

These questions are addressed across the remainder of the report: sub-questions 1, 2, and 5 are answered through the *Implementation* chapter and the ingestion analysis; sub-question 3 through the detection design and the *Analysis* of the signals it surfaces; and sub-question 4 through the analysis of archive-scale processing cost and its architectural consequences, drawn out in the *Discussion*.

3 Project Scope

The implementation specifically focuses on tanker vessels and is intended for the analysis and research of vessel behavior rather than as a live operational tracking service. The system ingests the Danish Maritime Authority's regional AIS feed both as ongoing daily batches published with a roughly three-day lag, so the data is continuously updated but never strictly real-time. Besides the daily batch we have an going historical backfill. The preprocessing pipeline mainly addresses data quality issues such as duplicate records, invalid coordinates, inconsistent timestamps, and incomplete vessel information.

3.1 Generative AI disclosure

In this project, AI was used as a supporting tool. It was used primarily for debugging and research, but also at times for refining terminology, suggesting phrasing, or exploring ideas and approaches. All substantive work, including the code implementations and analysis, was developed, reviewed, verified, and edited by the human author, who bears full responsibility for its accuracy and originality.

4 Related work

4.1 AIS-Based Vessel Behaviour Analysis and Maritime Monitoring

AIS data has become one of the most important sources for studying vessel behaviour, maritime traffic, and navigational safety. Because AIS continuously broadcasts a ship's position, speed, course, and identity, it allows researchers to analyse movement patterns and operational activities. As AIS datasets have grown larger and more complex, machine learning methods have become common in behaviour analysis. However, most studies assume that AIS data is already clean and structured, which is rarely true for large tanker archives that contain duplicates, missing values, and inconsistent timestamps.

Pallotta et al. introduced the TREAD framework, which uses unsupervised learning to extract traffic routes and detect anomalies [2]. Their work shows how AIS data can support route analysis and maritime awareness, but it relies on preprocessed trajectories and does not address the challenges of handling raw AIS data at national or multi-year scale.

Zhao applied machine learning to classify vessel trajectories and demonstrated that movement patterns can be identified effectively [6]. While the approach is useful for surveillance, it depends on AIS records that have already been filtered and organised into coherent tracks something that requires significant preprocessing in real-world tanker datasets.

Duan et al. used AIS motion features to classify operational phases such as cruising, anchoring, and berthing [7]. Their model achieved high accuracy, but only when the input data was consistent and reliable. Large-scale AIS archives often lack this consistency without strong cleaning and validation steps.

Newaliya et al. combined clustering with SHAP-based explainability to improve the transparency of AIS-based models [8]. Their work highlights how individual AIS features influence behaviour patterns, but explainability is only meaningful when the underlying data is trustworthy.

Overall, the literature shows that AIS data is powerful for behaviour analysis, anomaly detection, and monitoring. But a major gap remains: most studies depend on already-clean AIS data and do not address the engineering work required to ingest, clean, and manage raw AIS archives at scale. This thesis fills that gap by developing an automated ELT-based pipeline that prepares tanker AIS data for large-scale analysis and makes these behavioural methods realistically applicable.

4.2 Maritime Energy Efficiency and Operational Optimisation

Environmental sustainability and operational efficiency have become major focus areas in maritime research. AIS data is now frequently combined with operational and environmental information to support fuel-consumption prediction, voyage optimisation, and emission-reduction strategies. These studies show how AIS can be used beyond traffic monitoring, extending into energy efficiency and environmental performance.

Vergara et al. proposed a framework that reduces environmental impact through voyage segmentation and propulsion-power optimisation [9]. Their work demonstrates that AIS data can support operational decision-making aimed at improving vessel efficiency. However, the framework depends on well-structured operational datasets and does not address the challenges of preparing large-scale AIS archives for such optimisation tasks.

Uyanık et al. evaluated several machine learning models including ANN, DNN, Bayesian Regression, and XGBoost to estimate ship power requirements and fuel consumption using real voyage data [10]. Their results highlight the potential of data-driven models for energy-efficient operations, but the study assumes that AIS and sensor data are already cleaned, synchronised, and aligned, which is rarely the case in raw tanker datasets.

Zhang et al. extended this line of work by proposing a Bi-LSTM model with an attention mechanism for predicting fuel consumption under real operational conditions [11]. The model achieved strong performance using large-scale datasets that included weather, voyage reports, and sensor measurements. Yet, similar to other studies, the approach relies on high-quality, preprocessed data and does not consider the engineering effort required to integrate AIS with heterogeneous operational sources.

Beyond predictive modelling, maritime research has also emphasised speed optimisation and emission reduction. Psaraftis and Kontovas showed that vessel speed has a direct impact on fuel consumption and greenhouse-gas emissions, and that operational optimisation strategies can significantly improve environmental performance [12]. Their findings reinforce the importance of reliable movement and operational data when designing optimisation systems.

Overall, the reviewed studies demonstrate that AIS data, when combined with operational information, can effectively support fuel-efficiency improvements and emission-reduction strategies. However, a common limitation across this literature is the assumption that AIS and operational datasets are already clean, consistent, and analysis-ready. This thesis addresses that gap by developing a dynamic (DYN) architecture that ingests, validates, and integrates tanker AIS data with operational information, enabling these energy-efficiency and optimisation models to be applied reliably at scale.

4.3 Explainable Artificial Intelligence in Maritime Research

The use of machine learning in maritime research has grown rapidly, but it has also raised concerns about how transparent and understandable these models are. Even though many models achieve high accuracy, it is often difficult to understand how they arrive at their predictions. This is why transparency and interpretability have become important topics in the field.

Guidotti et al. presented a detailed survey of methods for explaining black-box machine learning models [13]. They emphasised that explainability is essential for user trust, accountability, and understanding how a model makes decisions. In other words, when a system generates a prediction, users should be able to understand which features or patterns influenced that output.

In the maritime domain, Newaliya et al. demonstrated how explainability techniques such as SHAP can be applied to AIS-based analytical models [8]. Their work showed that explainable machine learning can make vessel movement patterns easier to interpret and can improve the transparency of model outputs.

Explainability is becoming increasingly important because many maritime applications are safety-critical, such as anomaly detection, vessel monitoring, and decision-support systems. In these cases, accuracy alone is not enough users also need to understand why a prediction was made.

Overall, the literature shows that explainability will play a growing role in future maritime machine learning systems. In this thesis, explainability is discussed only as a research trend; the proposed DYN system does not implement XAI, so the topic is kept at a conceptual level.

4.4 Machine Learning Applications and Challenges in AIS-Based Maritime Research

The increasing availability of AIS data has accelerated the use of machine learning across many maritime applications. Yang et al. presented a comprehensive review of AIS-based machine learning research, covering key areas such as vessel trajectory prediction, collision avoidance, anomaly detection, maritime safety, and energy efficiency [14]. Their review shows that machine learning has become a central component of modern maritime analytics.

A major point highlighted in the review is that AIS datasets often suffer from data quality issues. Common problems include missing values, duplicate messages, irregular or noisy trajectories, inconsistent formats, signal gaps, and outliers. If these issues are not handled properly during preprocessing, they can significantly reduce the accuracy and reliability of machine learning models. This remains one of the biggest limitations in applying ML to real-world AIS data.

The authors also discussed the growing use of both traditional and deep learning techniques, such as Random Forests, Support Vector Machines, LSTM networks, CNNs, and various clustering algorithms. These methods are widely used for behaviour analysis, prediction, and pattern recognition. However, all of these models depend heavily on the quality and consistency of the input data.

Overall, the reviewed literature clearly shows that high-quality AIS preprocessing is a critical prerequisite for successful maritime analytics. The reliability, accuracy, and usefulness of machine learning applications depend directly on how well AIS data is cleaned, validated, and structured before analysis. This makes data preparation one of the most important foundations for any AIS-based analytical system.

4.5 Research Gap

The reviewed literature shows that AIS data is widely used for vessel behaviour analysis, anomaly detection, operational phase identification, route optimisation, fuel consumption prediction, and maritime decision support [2], [6], [7], [14]. With the growth of machine learning and deep learning, researchers now have strong analytical tools for extracting patterns and making predictions from AIS data.

Table 1 summarises the focus, contributions, and limitations of the reviewed studies.However, most existing studies focus mainly on the analytical and predictive side of AIS research. They usually assume that AIS data has already been collected, cleaned, structured, and prepared for modelling. Very limited attention is given to the engineering challenges that come before analysis such as acquiring large AIS streams, cleaning noisy data, handling duplicates and gaps, storing multi-year datasets, and delivering them through scalable software systems.

Only a small number of studies discuss the development of complete AIS data infrastructures that include automated processing pipelines, database systems, APIs, and web-based visualisation tools. Yet, such infrastructure is essential for real-world maritime analytics, especially when working with large-scale tanker datasets that require continuous ingestion and reliable data delivery.

To address this gap, the present project develops a dynamic AIS tanker monitoring platform that integrates automated data processing, PostgreSQL-based storage, REST API services, and interactive web visualisation. The goal is to improve AIS data accessibility, support efficient data management, and provide a practical foundation for future maritime analytics, anomaly detection, and vessel monitoring applications.

Table 1: Summary of Key Studies in AIS-Based Maritime Research

Study	**Focus Area**	**What They Did**	**Limitation / Gap**
Pallotta et al. (TREAD)	Route extraction and anomaly detection	Used unsupervised learning to extract maritime routes and detect anomalies	Relies on preprocessed trajectories; does not address large-scale raw AIS cleaning
Zhao	Trajectory classification	Applied ML to classify vessel movement patterns	Depends on filtered and organised AIS tracks; preprocessing challenges not addressed
Duan et al.	Operational phase detection	Classified phases such as cruising, anchoring, and berthing using AIS motion features	Requires consistent and reliable AIS data; large-scale archives often noisy
Newaliya et al.	Clustering + SHAP explainability	Combined clustering with SHAP to interpret AIS behaviour patterns	Explainability meaningful only when AIS data is already clean and trustworthy
Vergara et al.	Voyage optimisation	Proposed framework for propulsion-power optimisation using AIS	Assumes structured operational datasets; no handling of raw AIS archives
Uyanık et al.	Fuel consumption modelling	Evaluated ANN, DNN, Bayesian Regression, XGBoost for power estimation	Assumes AIS and sensor data are already cleaned and synchronised
Zhang et al.	Fuel prediction (Bi-LSTM)	Used Bi-LSTM with attention for fuel consumption prediction	Relies on high-quality, preprocessed multi-source datasets
Psaraftis & Kontovas	Speed–emission optimisation	Showed relationship between vessel speed and emissions	Requires reliable movement data; does not address AIS data quality issues
Yang et al.	ML review for AIS	Reviewed ML applications in AIS-based maritime research	Highlights data quality issues but does not propose data infrastructure solutions

5 Project-Related Theory

This section sets out the concepts that the rest of the chapter and the Implementation chapter build on: the Automatic Identification System as a data source, the data-engineering pattern used to handle it, and the orchestration model used to drive that pattern on a schedule.

The Automatic Identification System. AIS is a vessel radio reporting system originally introduced for collision avoidance and traffic monitoring. Under the IMO's SOLAS Convention (Chapter V, Regulation 19), AIS transmitters are mandatory on vessels of 300 gross tonnage and above engaged on international voyages, on all cargo vessels of 500 gross tonnage and above, and on all passenger ships. Each transponder broadcasts on dedicated VHF channels, and the resulting messages are received by other ships, by coastal base stations, and by satellites. As a result, very large volumes of AIS data are collected every day and are used to monitor vessel movements, support navigation safety, and analyze maritime traffic [4]. The heterogeneous data produce the national AIS data record used in this work [1].

AIS messages fall into three logical categories. *Static* information identifies the vessel itself its MMSI (Maritime Mobile Service Identity), IMO number, name, callsign, type, and physical dimensions and changes rarely. *Dynamic* information describes the vessel's present state of motion position (latitude/longitude), speed over ground (SOG), course over ground (COG), heading, rate of turn, and navigational status and is broadcast at intervals from a few seconds (underway) to several minutes (at anchor). *Voyage-related* information draught, destination, and estimated time of arrival (ETA) is entered by the crew and updated only when it changes. Because vessel identity and vessel movement are reported in separate message types from the same transponder, either field can be inconsistent, stale, or absent in any given record, and the literature treats AIS as a high-volume but quality-variable source [4], [5].

Extract-Load-Transform as a handling pattern. The volume and variability of AIS make the traditional Extract Transform Load (ETL) approach clean records in the loader and write only the clean version to the database a poor fit: any defect in the cleaning logic destroys data that cannot be cheaply re-fetched. The pattern adopted here is therefore the inverse, *Extract-Load-Transform* (ELT): raw records are first loaded *unmodified* into a staging table inside the database, and only afterwards are they validated, typed, and projected into the analytical tables. The staging layer acts as an immutable system of record from which any subsequent transformation can be re-derived, and it is what makes the pipeline re-runnable without going back to the source. In other words; *ETL* is destructive, while *ELT* preserves integrity. [15]

Workflow orchestration. Because AIS .Zip are published inconsistently on a daily basis by our Data-Provide [16], ingestion logic itself is not enough, the system should not only expect to ingest them continuously, it must be *driven* on a schedule, with retries, with visibility into what ran and what failed. Apache Airflow provides this through directed graphs (DAGs) of typed tasks, in which each task is a self-contained unit of work and the DAG expresses the dependencies between them. The orchestration tier of this project is built on three such DAGs; their concrete responsibilities are documented in the Implementation chapter.

6 Methodology

The project follows a *design science research* approach: knowledge is produced by designing, building, and evaluating a software artifact that responds to an identified real-world problem. The problem is that large-scale AIS data is difficult to access, messy, and hard to interpret without specialist tools. The artifact itself is the primary research contribution, and its design choices are evaluated against the research objectives stated in the Introduction.

Because the requirements were not fully known in advance and were expected to evolve as we learned more about the dataset, the work proceeded iteratively rather than against a fixed, plan-driven model: our vision of the solution took shape across successive build-and-evaluate cycles, ultimately producing an end-to-end platform spanning ingestion, storage, preprocessing, programmatic access, and visualization. Each iteration produced a working vertical slice of functionality that could be inspected and built upon, allowing design assumptions to be tested early and revised cheaply. The early prototype that confirmed the source layout and the available AIS fields was carried out in a Python notebook; once the fields were understood and the download URL pattern was confirmed, the validated logic was reimplemented in the production stack rather than left as notebook code.

Unfortunately, this approach also meant that some work was left behind, such as *trajectory* calculation and *voyage* prediction. We solved these problems early on in a Python notebook, but never ported them into the PostgreSQL/ASP.NET/React/Airflow/MapLibreGL environment.

6.1 Development Workflow

Development was carried out by a two-person team and organized around sprint-based milestones, managed on a Scrum board [17] on which each sprint's work was broken down into smaller items implemented in parallel. An initial planning artifact divided the work into two broad phases: a *data setup* phase, concerned with sourcing, ingesting, cleaning, and storing AIS data, and a subsequent *prototyping* phase intended to build features on top of the resulting foundation. Within these phases, individual sprints targeted concrete deliverables: the ingestion routine, the database schema, the REST API's controllers, the frontend pages, and the anomaly detectors.

Version control was used not only for collaboration but as a continuous record of the project's evolution. The complete commit history documents the progression from early exploratory notebooks to the deployed system. Furthermore, Git served as the mechanism for deployment: changes pushed to the repository were pulled directly onto the production server, making the commit history and the deployed state one and the same.

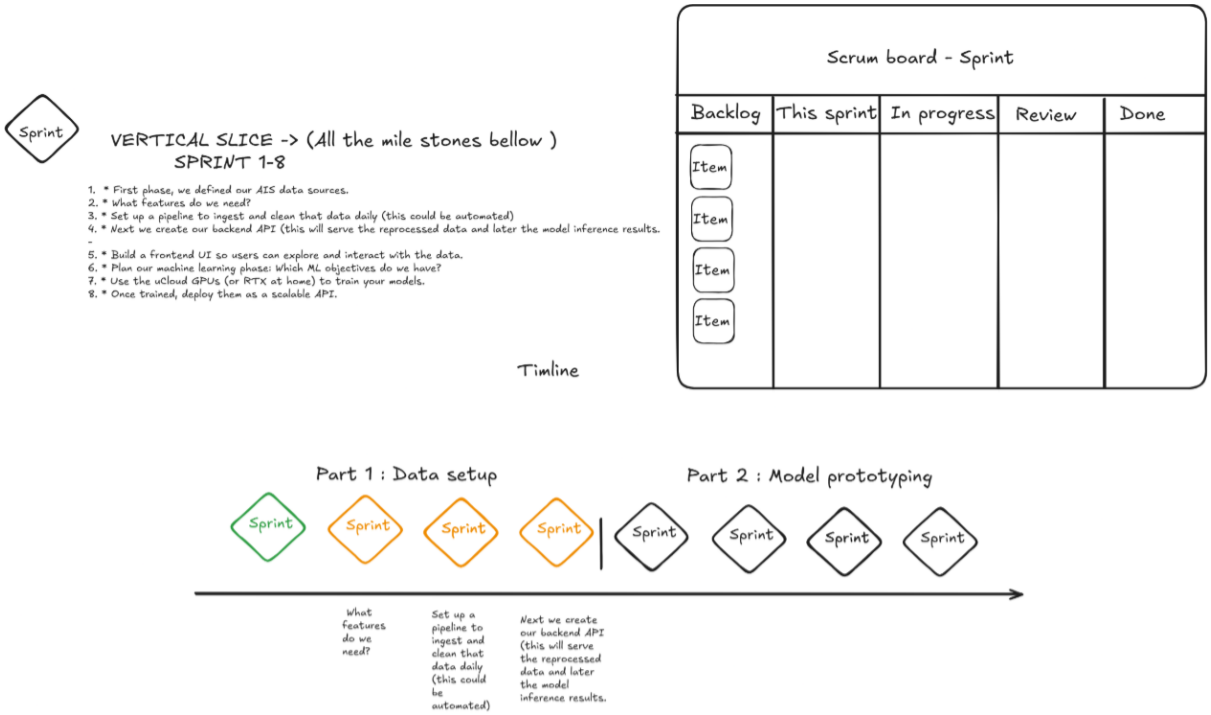


Figure 1: Iterative development workflow and sprint-based project planning.

6.2 System Architecture Overview

The platform is organized as four cooperating tiers; an *orchestration* tier of Apache Airflow DAGs, a *storage and processing* tier built on PostgreSQL, a *service* tier exposing an ASP.NET Core REST API, and a *presentation* tier rendering vessels on a MapLibre GL map sketched in Figure 2. Their detailed construction, and the narrow contracts that keep the tiers decoupled, is the subject of the Implementation chapter (Section 7.2). The purpose here is to show where each piece of work sits and how data flows.

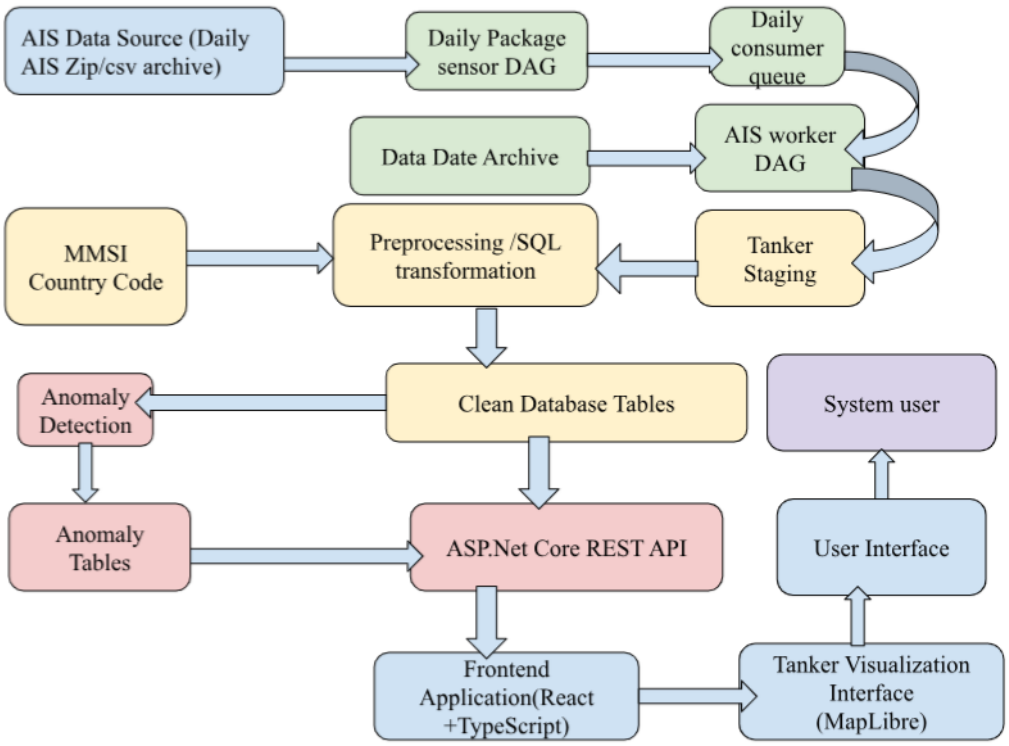


Figure 2: Overall system architecture of the AIS tanker processing platform.

6.3 Dataset Introduction

The data used in this project was obtained from the openly available historical AIS archive published by the Danish Maritime Authority through its public AIS data service (`https://aisdata.ais.dk`) [16]. It provides daily archive files covering Danish waters and imposes no licensing barrier to academic use; this is our justification for choosing this source over a commercial alternative. Files are published *retrospectively* with a roughly three-day lag, which means that the most recent day available at any given moment is not yesterday but the day three days earlier; the ingestion logic accommodates this by targeting an offset of three days.

Initial download tests confirmed that a single daily AIS archive could exceed 500 MB compressed, expanding to over 3 GB once extracted. Volumes of this magnitude made manual downloads impractical and grounds for an automated, memory-bounded ingestion workflow.

6.4 Data Description and Explanation

Building on 5. The primary vessel identifier is the MMSI, a nine-digit number assigned per transmitter; its first three digits the Maritime Identification Digits (MID) encode the flag country and are the basis for the flag-derivation step used elsewhere in this work. The IMO number, where present, is a permanent identifier for the hull itself and, unlike

the MMSI, does not change when a vessel changes flag or operator [4]; this divergence between a stable hull identifier and a mutable transmitter identifier is precisely what the identity-irregularity detection later exploits.

For this project, the `Ship type` attribute was used to filter the dataset to tanker vessels specifically. Exploratory analysis confirmed that the field contains a small, well-defined set of categories and that "Tanker" is represented as a single consistent value, allowing reliable filtering.

The complete set of AIS columns available in the source files is given in Table 2 bellow.

Table 2: AIS columns available in the Danish Maritime Authority source files. Positional fields (Latitude, Longitude) use the WGS-84 datum.

No.	Column Name	Format / Description
1	Timestamp	Timestamp from AIS basestation (e.g. 31/12/2015 23:59:59)
2	Type of mobile	Type of AIS target (Class A vessel, Class B vessel, etc.)
3	MMSI	MMSI number of vessel
4	Latitude	Latitude coordinate (e.g. 57.8794)
5	Longitude	Longitude coordinate (e.g. 17.9125)
6	Navigational status	Vessel navigation status from AIS message
7	ROT	Rate of turn from AIS message
8	SOG	Speed over ground
9	COG	Course over ground
10	Heading	Vessel heading
11	IMO	IMO vessel number
12	Callsign	Vessel callsign
13	Name	Vessel name
14	Ship type	AIS ship type
15	Cargo type	Cargo type from AIS message
16	Width	Vessel width
17	Length	Vessel length
18	Type of position fixing device	Position fixing device type
19	Draught	Vessel draught
20	Destination	Vessel destination
21	ETA	Estimated time of arrival
22	Data source type	Source type (e.g. AIS)
23	Size A	Distance from GPS to bow
24	Size B	Distance from GPS to stern
25	Size C	Distance from GPS to starboard side
26	Size D	Distance from GPS to port side

The remaining construction of the platform how this raw dataset is downloaded, staged, cleaned, indexed, exposed through an API, and rendered on a map is documented in the Implementation chapter next.

7 Implementation

This chapter describes how the platform was built. It follows the life of a single AIS batch from the moment its availability is detected at the source, through staging, transformation and anomaly detection in the database, to its exposure over a versioned REST API and its rendering on an interactive map.

Sections 7.1 Show our hard-disk partitions. Section 7.2 gives the component overview. Sections 7.3–7.7 cover the data pipeline (orchestration, ingestion, transformation, schema, detection); and Sections 7.8–7.9 cover the delivery tier (the .NET backend and the React frontend). The rationale behind these choices, and the trade-offs they entail, is examined separately in the Discussion.

7.1 ZFS pool & RAIDZ1

```
sebastian@k8s-worker-node002: ~
$ zpool status datapool
  pool: datapool
 state: ONLINE
  scan: scrub repaired 0B in 00:02:07 with 0 errors on Sun May 10 2026
config:
        NAME        STATE     READ WRITE CKSUM
        datapool    ONLINE       0     0     0
          raidz1-0  ONLINE       0     0     0
            sdb     ONLINE       0     0     0
            sdc     ONLINE       0     0     0
            sdd     ONLINE       0     0     0
            sde     ONLINE       0     0     0
errors: No known data errors

$ zpool list -v datapool
NAME         SIZE  ALLOC   FREE   FRAG    CAP  HEALTH
datapool    3.62T   355G  3.28T    10%     9%  ONLINE
  raidz1-0  3.62T   355G  3.28T    10%  9.55%  ONLINE
    sdb      932G      -      -      -      -   ONLINE
    sdc      932G      -      -      -      -   ONLINE
    sdd      932G      -      -      -      -   ONLINE
    sde      932G      -      -      -      -   ONLINE
```

Figure 3: The `datapool` ZFS storage substrate: a single RAIDZ1 vdev over four 932 GB drives (`sdb`–`sde`), reporting 3.62 TB raw pooled capacity at 9% utilisation.

Storage substrate: the ZFS pool. The platform's data lives on a ZFS pool (datapool) rather than a single formatted disk. ZFS combines a filesystem and a volume manager:

physical disks are grouped into one or more virtual devices (vdevs), from which the pool draws its capacity. The pool here is a single RAIDZ1 vdev spanning four drives (sdb–sde).

The single-parity layout tolerates any one-disk failure; the 1.8 TB ingestion ceiling (Section 5.2) sits below usable capacity to keep the allocator in its low-fragmentation regime. This means the pool have plenty of free space to ensure high I/O performance.

Figure 4: The physical `datapool` volumes: four 932 GB SATA drives wired through a USB-to-SATA adapter to a repurposed laptops with Linux Ubuntu Server operating system. These are the same four drives (`sdb`–`sde`) reported `ONLINE` in the RAIDZ1 vdev of Figure 3.

7.2 Component overview and technology stack

The system is a full-stack data platform composed of four cooperating tiers, sketched in Figure 2: (i) an *orchestration tier* of Apache Airflow DAGs that schedules and drives ingestion; (ii) a *storage and processing tier* built on PostgreSQL, where raw data is staged, transformed in place, and screened for anomalies; (iii) a *service tier* implemented as an ASP.NET Core REST API over Entity Framework Core; and (iv) a *presentation tier*, a React/TypeScript single-page application rendering vessels on a MapLibre GL map. The tiers are deliberately decoupled: each communicates with its neighbour through a narrow, explicit contract a database schema between processing and service, and a versioned JSON API between service and presentation so that any one tier can evolve independently. The whole stack is containerised with Docker Compose. Table 3 records the principal technologies and versions.

Table 3: The solution full-stack technologies.

Tier	Component	Version
Orchestration	Apache Airflow (TaskFlow API)	2.9.3
	pandas, psycopg2	2.1.4, 2.9.9
Storage	PostgreSQL	16-alpine
Service	.NET / ASP.NET Core	8.0
	Entity Framework Core + Npgsql	8.0.10
	Asp.Versioning.Mvc	8.1.0
	Swashbuckle (Swagger/OpenAPI)	6.6.2
Presentation	React + TypeScript + Vite	19.2.5, 6.0.2, 8.0.9
	MapLibre GL, axios, React Router	5.23.0, 1.15.1, 7.14.1
Packaging	Docker Compose	v5.1.3

7.3 Orchestration: the Airflow DAGs

Ingestion is driven by three Airflow DAGs with distinct responsibilities, written with the TaskFlow (`@dag`/`@task`) API. They are separated by how often they run and how expensive they are, so that cheap, frequent work is never blocked behind rare, costly work.

The daily sensor (`daily_package_sensor`). Because the source publishes each day's archive retrospectively, this DAG targets the date three days prior. A `PythonSensor` polls the source with an HTTP `HEAD` request every five minutes (within a nine-hour window) until the file appears, at which point a single high-priority job (`priority = 100`, `requester = 'daily'`) is enqueued into the `data_consumer_queue` table. The sensor only *detects and enqueues*; it does no downloading itself, keeping the trigger lightweight.

The worker (`ais_worker`). The worker is the heart of the pipeline. It runs every fifteen minutes with `max_active_runs = 1` (a single concurrent run) and `retries = 0`, since failures are recorded in the queue rather than silently retried. Each run executes a linear seven-task pipeline, each task passing a small job dictionary to the next:

1. `claim_next_job` selects the highest-priority pending job and marks it `in_progress`;
2. `check_already_done` skips (and marks `done`) if the date already exists in the archive;
3. `check_budget` aborts if the database has reached the 1.8 TB storage ceiling;
4. `download_and_stage` fetches, unzips and bulk-loads the batch into `tanker_staging`;
5. `promote` runs the transformation SQL and captures the number of positions inserted;

6. `detect_anomalies` runs the lightweight, idempotent invalid-MID detector over the `tankers` dimension.;

7. `finalize` writes the archive row and marks the queue entry `done`.

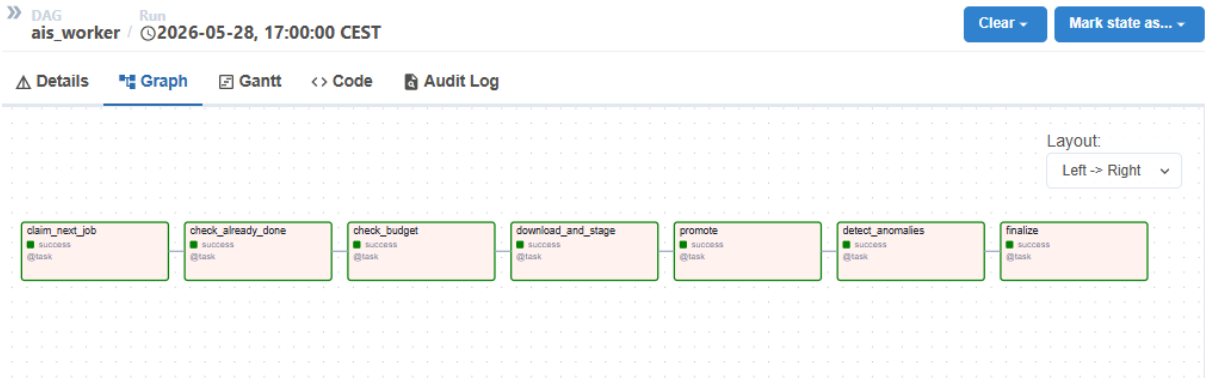


Figure 5: DAG graph view of the `ais_worker` workflow in Apache Airflow. The pipeline consists of seven sequential tasks explained above.

Each task opens and closes its own database connection, so a failure leaves no dangling transaction and the queue row is left in a recoverable state.

The weekly anomaly scan (`weekly_mmsi_swap`). The most expensive detector a whole-history aggregation is isolated in its own DAG scheduled for 04:00 every Sunday, precisely so that its cost (documented in the Analysis) never delays daily ingestion. This separation of heavy, global detection from light, incremental detection is the operational expression of the decoupling argued for in the Discussion.

7.4 Ingestion and staging

The `download_and_stage` task first issues a `HEAD` probe to distinguish a genuinely missing source day (HTTP 404, handled as a clean skip) from a transient error. On success it streams the archive into memory, opens the contained CSV, and reads it in fixed-size chunks of 20,000 rows. Reading in chunks bounds memory use independently of file size daily archives can exceed half a gigabyte and each chunk is written with PostgreSQL's bulk `COPY` path via `copy_expert`, which is markedly faster than row-by-row inserts. Column headers are normalised (stripping the leading "`#`" and whitespace) and renamed to the staging schema, and each row is tagged with its source filename and batch date for provenance (Listing 1).

```
z = zipfile.ZipFile(io.BytesIO(r.content))
csv_name = z.namelist()[0]

with z.open(csv_name) as f:
    for chunk in pd.read_csv(f, sep=',', chunksize=20000, low_memory=
        False):
        chunk.columns = chunk.columns.str.replace('#', '', regex=
            False).str.strip()
        chunk = chunk.rename(columns=COL_MAP)
        chunk['source_batch_date'] = str(target_day)

        buf = io.StringIO()
        chunk.to_csv(buf, index=False, header=False)
        buf.seek(0)
        with conn.cursor() as cur:\textbf{}
            cur.copy_expert(
                f"COPY tanker_staging ({','.join(chunk.columns)})
                    FROM STDIN WITH CSV",
                buf)
        conn.commit()
```

Listing 1: Bounded-memory, chunked bulk ingestion into the staging table (abridged).

At this stage every field is stored as raw text in `tanker_staging`; no parsing or validation has occurred. This preserves the source verbatim as an immutable system of record from which all derived state can be recomputed [15], and defers all interpretation to the transformation step, so that a parsing rule can be changed and re-applied without re-downloading.

7.5 Transformation: the staging-to-clean ETL

Transformation is implemented entirely in SQL and executed in the database, so no large intermediate result is ever materialised in the application. The script (run by the `promote` task, parameterised by `target_day`) performs two passes (Listing 2).

The *first pass* selects tanker rows with a syntactically valid seven-digit IMO, normalises their numeric fields (converting comma decimals and coercing empty strings to `NULL`), derives the flag state by joining the MMSI prefix against the MID lookup table (falling back to the sentinel `UN`), and upserts them into the `tankers` dimension keyed on IMO. Positions for these known tankers are then inserted with coordinate-range validation and an `ON CONFLICT DO NOTHING` clause against a uniqueness constraint that silently absorbs duplicate reports.

The *second pass* handles tanker rows whose IMO is missing or malformed. Rather than discarding them, it inserts their positions with `imo_status = 'unknown'` and `anomaly_flag = TRUE`, deliberately preserving degraded or potentially deceptive identity data for later inspection. The script returns the total number of positions inserted, which the worker records in the archive.

```sql
-- Pass 1: known tankers -> upsert dimension, flag derived from MMSI MID prefix
INSERT INTO tankers (imo, mmsi, vessel_name, ship_type, flag, ...)
SELECT DISTINCT ON (TRIM(s.imo))
       TRIM(s.imo), TRIM(s.mmsi), TRIM(s.vessel_name), TRIM(s.ship_type),
       COALESCE(mcc.country_code, 'UN')
FROM tanker_staging s
LEFT JOIN mmsi_country_codes mcc ON LEFT(TRIM(s.mmsi), 3) = mcc.mid_code
WHERE s.source_batch_date = %(target_day)s
  AND LOWER(TRIM(s.ship_type)) = 'tanker'
  AND TRIM(s.imo) ~ '^[0-9]{7}$'
ON CONFLICT (imo) DO UPDATE SET mmsi = EXCLUDED.mmsi, flag = EXCLUDED.flag, ... ;

-- Pass 2: missing/unknown IMO -> preserved, not dropped, flagged anomalous
INSERT INTO tanker_positions (tanker_id, raw_imo, imo_status, anomaly_flag, ...)
SELECT NULL, TRIM(s.imo), 'unknown', TRUE, ...
FROM tanker_staging s
WHERE s.source_batch_date = %(target_day)s
  AND LOWER(TRIM(s.ship_type)) = 'tanker'
  AND (s.imo IS NULL OR TRIM(s.imo) = '' OR LOWER(TRIM(s.imo)) = 'unknown')
  AND REPLACE(s.latitude_raw, ',', '.')::DOUBLE PRECISION BETWEEN -90 AND 90
  AND REPLACE(s.longitude_raw, ',', '.')::DOUBLE PRECISION BETWEEN -180 AND 180;
```

Listing 2: The two-pass transformation: upsert known tankers with derived flag (pass 1), preserve unknown-IMO positions as anomalous (pass 2). Abridged.

7.6 Database schema and integrity

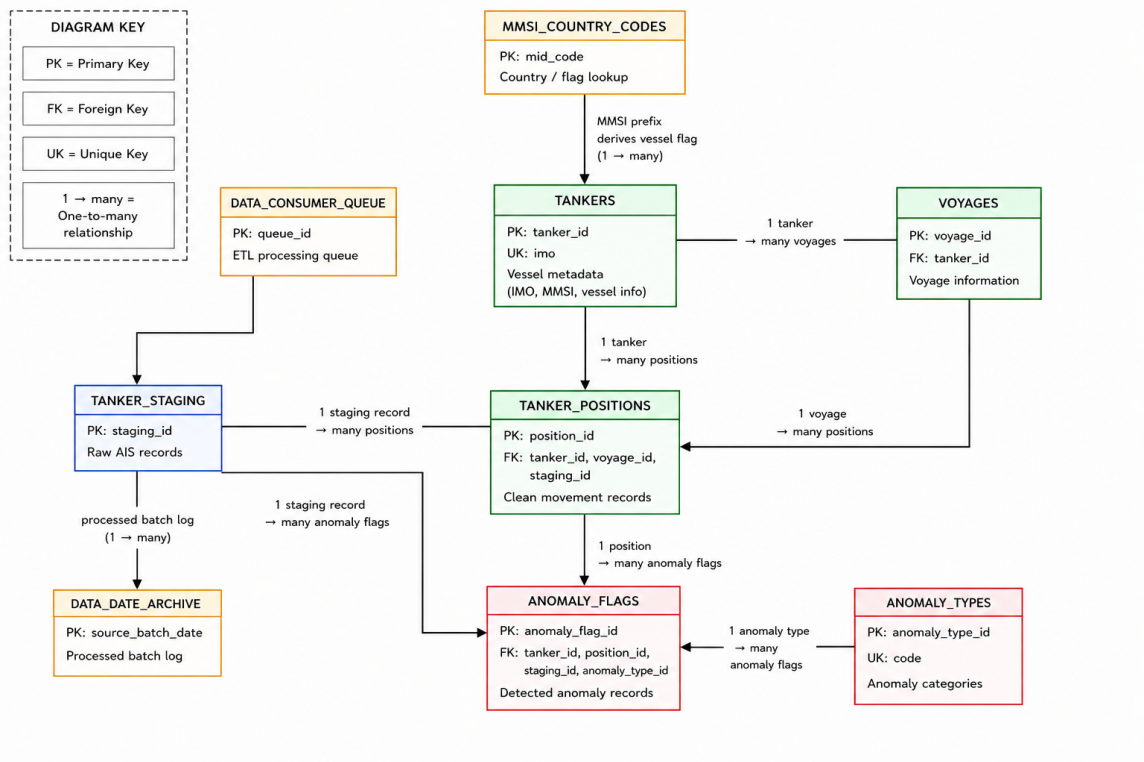


Figure 6: Database schema of the AIS tanker processing platform.

Figure 6 illustrates the database structure used in the AIS tanker processing platform...

The schema comprises ten tables (Table 4). It is organised around a small `tankers` dimension one row per vessel, keyed by an internal `tanker_id` surrogate and constrained `UNIQUE` on IMO as the natural external identifier and a large `tanker_positions` fact table referencing it by foreign key. Referential integrity is enforced with cascade and set-null rules appropriate to each relationship.

Two integrity mechanisms are worth highlighting. First, duplicate position reports are suppressed at the database level by a *partial unique index* on `(tanker_id, timestamp_utc, latitude, longitude)` restricted to rows with a known `tanker_id`; combined with the `ON CONFLICT` clause above, this makes re-ingestion of a day idempotent. Second, the `anomaly_types` catalogue is seeded with the recognised anomaly categories (invalid IMO, unknown IMO, invalid MID, invalid position, missing flag, missing draught, missing destination, and MMSI swap), so that detected anomalies reference a controlled vocabulary rather than free text. Indexing is applied selectively under the storage budget composite and partial indexes on the columns actually queried (e.g. `(tanker_id, timestamp_utc DESC)`) rather than blanket coverage.

Table 4: The ten persistent tables and their roles.

Table	Role
`tanker_staging`	Raw, unparsed AIS rows as ingested (system of record)
`tankers`	Cleaned vessel dimension; `UNIQUE(imo)`, derived flag
`tanker_positions`	Validated position fact table; FK to `tankers`
`voyages`	Voyage segmentation (enrichment, partly future work)
`tracked_tankers`	Watchlist of vessels of interest
`anomaly_types`	Controlled vocabulary of anomaly categories
`anomaly_flags`	Detected anomalies, FK to type / vessel / position
`data_consumer_queue`	Priority work queue for ingestion jobs
`data_date_archive`	Per-batch processing record and counts
`mmsi_country_codes`	MID-prefix→country lookup for flag derivation

7.7 Anomaly detection

Detection is implemented as a set of idempotent SQL scripts, decoupled from ingestion so they can be re-run and extended without side effects. Each script guards its inserts with a `NOT EXISTS` predicate against `anomaly_flags`, so repeated execution never double-counts. Three detectors were implemented, spanning the cost spectrum:

- **Invalid MID (`INVALID_MMSI_MID`)** the lightweight per-batch detector run by the worker. It flags vessels whose derived flag is `UN`, i.e. whose MMSI prefix resolves to no assigned national authority. It operates only on the small `tankers` dimension, so it is cheap enough to run on every ingestion.
- **MMSI swap (`MMSI_SWAP`)** the weekly whole-history detector. It groups the staging history by IMO and flags any IMO observed broadcasting more than one distinct MMSI, the canonical footprint of identity manipulation. Its cost (analysed in the Analysis) is why it is isolated to a weekly schedule.
- **Invalid position (`INVALID_POSITION`)** a day-parameterised detector that flags out-of-range coordinates surviving in staging, completing the planned detection set.

7.8 Backend service and REST API

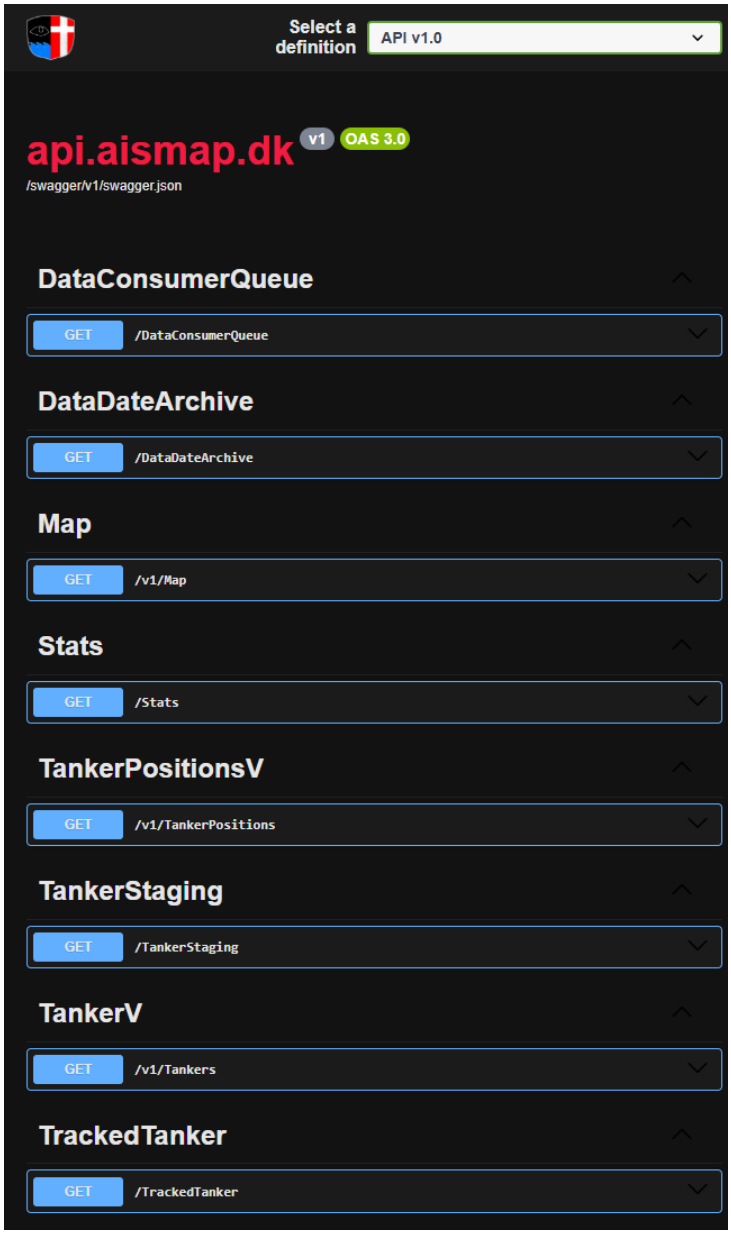


Figure 7: Swagger documentation for the AISMap API v1.0

The backend is separated into a data-access layer (`DataLayer`) and a web layer (`WebLayer`), with the ASP.NET Core host depending on both. The two layers communicate only through Data Transfer Objects, so the Entity Framework persistence entities are never serialised directly onto the wire [18]; every endpoint maps its domain model to a matching `DTO` before returning it. This keeps the public JSON contract stable even if the internal schema changes.

Data-access layer. The layer centres on an EF Core `DbContext` (`AisDB_Context`) whose `OnModelCreating` binds each `snake_case` PostgreSQL column to a PascalCase

property through explicit fluent configuration one mapping method per table, so the mapping for any given table is isolated and maintainable. All read paths use `AsNoTracking`, since the API is strictly query-only and change-tracking would be wasted overhead. Business queries are encapsulated behind an `IDataService` interface with a single `DataService` implementation, so controllers depend on an abstraction rather than on EF Core directly.

Two query shapes the ORM forced into the open. Two queries did not fit the LINQ-over-EF abstraction cleanly and are documented here because the workarounds are deliberate, not accidental. The first is the "latest position per active vessel" query that backs the map: it is expressed in raw SQL using PostgreSQL's `DISTINCT ON`/`LATERAL` (Listing 8) and projected onto a keyless entity type (`HasNoKey()`), because the LINQ provider cannot translate `DISTINCT ON`; a portable `GROUP BY` formulation was measurably slower at this row count. The second is pagination: on the multi-billion-row position table, the customary `COUNT(*)` that accompanies a paged response is an $O(n)$ scan, so the position endpoint omits it and returns `TotalItems = -1` as an explicit "not computed" sentinel rather than pay that cost on every request. The smaller `tankers` dimension, by contrast, is cheap to count, so its paged endpoint returns a true total — the count is computed where it is affordable and skipped where it is not.

Web layer and versioning. The web layer exposes resource-oriented controllers following the REST style [19], each decorated with the fixed-window rate limiter and inheriting a small `BaseController` that supplies URL generation. API versioning is negotiated from the URL segment, a query-string parameter, or a header (Listing 3), and the project uses two distinct versioning idioms depending on how far the two versions of a resource diverge. Where v2 only adds optional query parameters as for `/Map`, which gains a `sinceHours` window in v2 a single controller advertises both `[ApiVersion]`s and branches on the parameter. Where the two versions differ in behaviour as for `Tankers` and `TankerPositions`, whose v1 returns a fixed recent slice and whose v2 accepts paging and filtering they are split into separate `V1`/`V2` controller classes sharing one route template. This gives a simple, frozen v1 for casual callers and a richer v2 for analytical use, without either having to accommodate the other.

Surface and cross-cutting concerns. The running API (Figure 7) exposes the cleaned data and the pipeline's own operational state: vessel and position resources (`/Tankers`, `/TankerPositions`, `/Map`), the raw staging sample (`/TankerStaging`), the watchlist (`/TrackedTanker`), and three introspection endpoints over the orchestration tables (`/Stats`, `/DataConsumerQueue`, `/DataDateArchive`) so the queue depth and per-batch processing record described in Section 8.1 are themselves queryable over HTTP. A CORS policy restricts origins to the known frontend hosts, the fixed-window limiter caps request volume, and Swagger/OpenAPI documents both versions behind a dark-themed UI. The `/Stats` endpoint is a small but representative illustration of the project's central scaling constraint: it sums the per-batch counts recorded in `data_date_archive` rather than ag-

gregating the fact table, deriving fleet-wide totals without ever scanning the billions of rows they summarise.

```
builder.Services.AddApiVersioning(o =>
{
    o.DefaultApiVersion = new ApiVersion(1, 0);
    o.AssumeDefaultVersionWhenUnspecified = true;
    o.ReportApiVersions = true;
    o.ApiVersionReader = ApiVersionReader.Combine(
        new UrlSegmentApiVersionReader(),
        new QueryStringApiVersionReader("api-version"),
        new HeaderApiVersionReader("X-Version"));
});
```

Listing 3: API version negotiation from URL segment, query string, or header.

7.9 Frontend and visualisation

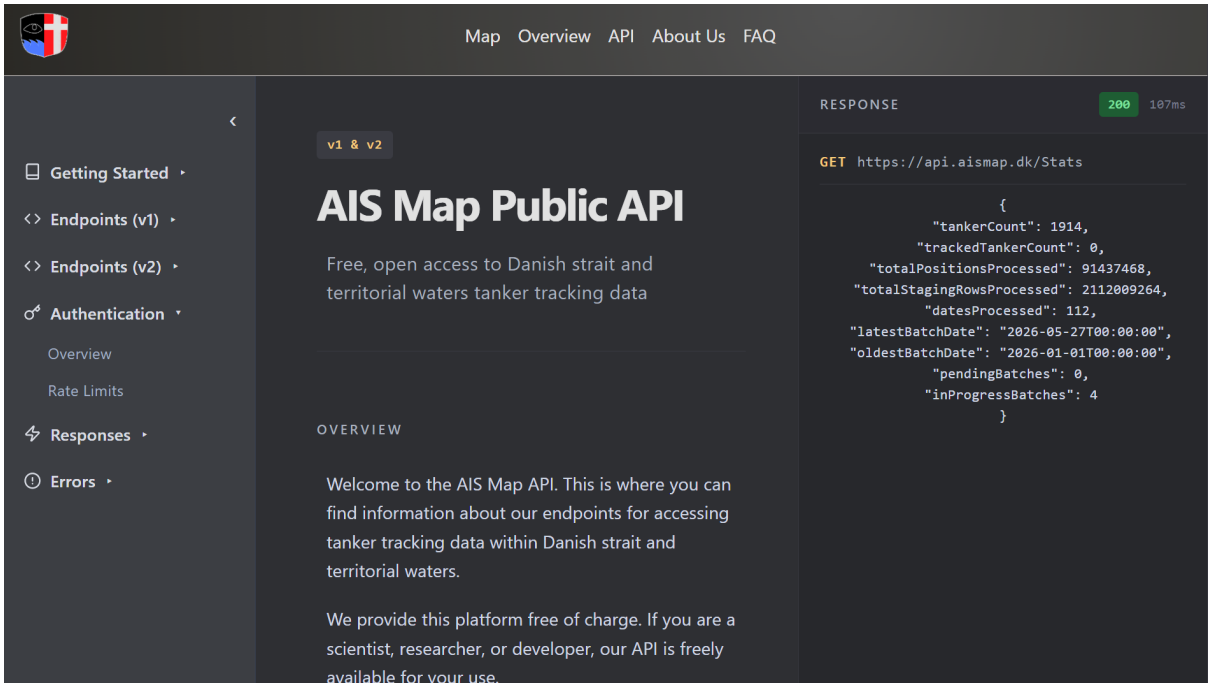


Figure 8: The custom API documentation page, a working client of the public API: the interactive "try it" panel issues a live `GET /Stats` request and renders the response.

The frontend is a React/TypeScript single-page application built with Vite and routed with React Router (a landing page, the map, an API documentation page, and supporting pages). Server responses are typed with TypeScript interfaces mirroring the DTOs, giving compile-time safety across the network boundary.

The map is built on a typed MapLibre GL wrapper component exposing markers, popups, viewport controls, and clustering. The map view fetches the latest position per active vessel from the API and renders one marker per vessel, colouring anomalous vessels

distinctly so that flag- and identity-irregular ships are immediately visible (Listing 4); clicking a marker opens a popup with the vessel's identity and movement attributes. The wireframe that guided this layout a central map with search, a vessel list, and contextual panels is shown in Figure 9.

```
useEffect(() => {
  fetch("https://api.aismap.dk/v1/Map")
    .then(res => res.json())
    .then((data: Vessel[]) => setVessels(data))
    .finally(() => setLoading(false));
}, []);

// ...
{vessels.map(v => (
  <MapMarker key={v.tanker_Id} longitude={v.longitude} latitude={v.
     latitude}>
    <MarkerContent>
      <div style={{ background: v.is_Anomalous ? "#ef4444" : "#3b82f6
         " }} />
    </MarkerContent>
  </MapMarker>
))}
```

Listing 4: Fetching latest positions and rendering colour-coded vessel markers (abridged).

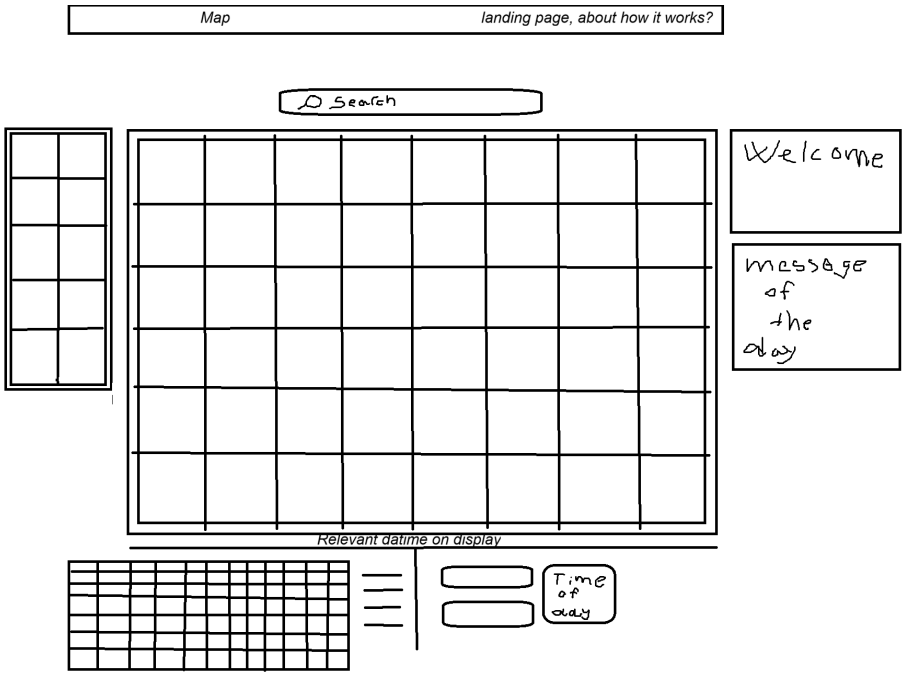


Figure 9: Frontend wireframe: a central map with a search bar, a vessel list, and contextual side panels, with a time-of-day control beneath the map.

Beyond the map, an API documentation page provides an interactive "try it" panel that issues live requests to each endpoint and renders the response, turning the documentation itself into a working client of the public API.

Lastly it is worth mentioning to maintain and realize our in-production system, we used a series of shell script: (`deploy.sh`, `apideploy.sh`, `frontdeploy.sh`) that each pull the latest commit, rebuild the relevant container with no-cache, and restart only that service. This is done so the API, frontend, and full stack can be redeployed independently, without interrupting other parts, like disturbing Airflow's ingestion.

8 Analysis

This chapter analyses the implemented system along four main elements: the data-reduction characteristics of the ingestion pipeline (Section 8.1); processing throughput and scalability (Section 8.2); the composition of the resulting tanker corpus (Section 8.3); and the anomaly signal that the pipeline surfaces (Section 8.4). Because the platform ingests daily uploaded packages (with a 3-day delay) and backfill of historical Danish AIS archive of the whole 2026, continuous to 2007, we have achieved a database with totalStagingRowsProcessed 2,139,117,650 → "two billion" raw position reports, we choose analysis to deliberately emphasis on quantities that remain computationally at this scale. Concretely, all figures reported here are either recorded *incrementally* at ingestion time (in the `data_date_archive` table) or derived from the comparatively small `tankers` dimension, rather than recomputed by full scans over the multi-billion-row fact table. The reasons for this constraint are themselves an empirical result and are returned to in Section 8.4 and Section 9.2.

8.1 The ingestion funnel and data reduction

Raw AIS is a high-volume, low-purity source: it interleaves all classes of mobile transmitter, contains malformed timestamps and coordinates, and carries substantial duplication. Following the *staging-then-clean* pattern from data-warehouse practice [20], the pipeline first preserves each batch verbatim in the `tanker_staging` table and only then applies transformation and validation, so that raw fidelity is never destroyed while the analytical tables remain compact. Each daily batch therefore passes through a *monotonic reduction*, summarised in Table 5:

1. **Staged rows** every transmitter of every mobile type, as received.
2. **Tanker rows** the subset whose `ship_type` normalises to `tanker`.
3. **Persisted positions** tanker rows that additionally pass coordinate-range and identifier validation and are written to `tanker_positions`.

The transition from stage 1 to stage 2 isolates the project's domain (tankers) from the dominant background traffic; the transition from stage 2 to stage 3 enforces the data-quality dimensions of *validity* (latitude $\in [-90, 90]$, longitude $\in [-180, 180]$) and *com-*

pleteness of the vessel identifier. Crucially, these per-stage counts are persisted at ingestion (Table 9), so the funnel can be quantified without ever counting the fact table directly.

Table 5: The ingestion funnel across all 109 processed batches (2026-01-01 to 2026-05-31). Compute the reduction factors as the ratio of adjacent stages.

Stage	Rows	Retained vs. previous
Staged (all mobiles)	2,139,117,650	—
Tanker-classified	186,126,788	9.1%
Persisted positions	92,409,639	47.7%

The proportion discarded between stages 2 and 3 is itself a data-quality finding: it quantifies how much nominally on-topic AIS is nevertheless unusable for spatial analysis, and so empirically justifies the validation step rather than presenting it as a formality.

8.2 Processing throughput and scalability

The ingestion worker reads each multi-hundred-megabyte daily file as a stream of fixed-size chunks (20,000 rows) and writes each chunk with PostgreSQL's bulk `COPY` path. This gives *bounded-memory* ingestion the working set is independent of file size and emits a running throughput figure (rows per second) to the task log. Reporting the mean and peak of this figure (Table 6) characterises the pipeline's practical capacity on the deployment hardware.

Table 6: Ingestion throughput, from worker task logs. "Per-day wall-clock" is the staging time for one daily archive.

Metric	Value
Mean ingest rate	≈5,585 rows/s
Peak ingest rate	≈5,700 rows/s
Per-day wall-clock (stage)	4,128 s (≈69 min)
Largest daily archive	30,156,066 rows

Two design choices bound throughput deliberately rather than accidentally. First, the worker DAG runs with `max_active_runs=1`, enforcing a single writer to avoid lock contention on the staging table. Second, an explicit storage-budget guard (a 1.8 TB ceiling on the ZFS pool) aborts ingestion before the database can outgrow the physical medium. Across the 110 batches recorded in `data_date_archive` to date, the pipeline achieves a mean staging rate of ≈5,585 rows/s and a per-day wall-clock of ≈69 min. Because every job is parameterised by a single date and is independent of the others, the design supports replay and out-of-order backfill. This property is actively exploited as the system continues to ingest the historical archive, mediated by the priority queue discussed in Section 9.1.

8.3 Composition of the tanker corpus

Once the dimension table `tankers` is populated, two characterisations of the fleet are available directly both produced at load time, neither requiring the heavy detection jobs. The first is the distribution of *flag states*, derived by mapping the first three digits of each MMSI (the Maritime Identification Digits, or MID) to an ITU-assigned country code; vessels whose prefix resolves to no known country are assigned the sentinel flag `UN`. The second is the `imo_status` split in `tanker_positions`, separating positions attributed to a validated seven-digit IMO from those whose IMO is absent or malformed.

```
-- Flag-state distribution (UN = MID prefix not in lookup table)
 un_vessels | total_vessels | un_pct
------------+---------------+--------
          0 |          1853 |    0.0

 flag | vessels
------+---------
 LR   |     231
 MT   |     213
 MH   |     212
 PA   |     121
 PT   |      95
 NO   |      88
 SG   |      80
 DK   |      76
 SL   |      75
 BS   |      62
 RU   |      62
 NL   |      59
 GR   |      57
 CM   |      57
 GB   |      52
 SE   |      45
 CY   |      44
 HK   |      26
 OM   |      23
 AG   |      22
 BB   |      21
 IT   |      15
 DE   |      14
 FR   |       9
 BE   |       9
 PW   |       9
 GQ   |       7
 FI   |       6
 KY   |       6
 TR   |       5
 -- 31 further flags with 1--4 vessels each (ID, VN, BM, GM, KM, KN,
 -- SA, LU, LV, ES, EE, CN, HR, GW, GY, AW, BD, MZ, TV, GN, SY, SC,
 -- IN, MY, VU, JP, MG, PH, CA, ST, PL)
(61 rows)
```

```
42
43 -- Validated vs unresolved IMO among persisted positions
44  imo_status | positions
45 ------------+-----------
46  unknown    |    411296
47  valid      |  88331118
```

Listing 5: Corpus composition queries (small dimension table and indexed status column).

Table 7: Fleet composition. Top flag states plus `UN` share; valid/unknown IMO split.

Category	Count
Distinct tankers (validated IMO)	1,867
Vessels flagged `UN` (unresolved MID)	0 (0.0%)
Positions with `imo_status = valid`	88,331,118
Positions with `imo_status = unknown`	411,296

The `UN` share is the proportion of the fleet whose MMSI prefix does not correspond to an assigned national maritime authority. This is a *precondition* for identity irregularity a necessary but not sufficient indicator and is interpreted cautiously in Section 9.3.

8.4 The anomaly signal

The system records two anomaly signals that are produced *within* the ordinary ingestion pass and therefore scale with it. (i) Positions carrying an absent or malformed IMO are still preserved rather than silently dropped and are written with `anomaly_flag = TRUE`, so that deceptive or degraded identity data is retained for inspection instead of being lost. (ii) Vessels with an unresolved MID prefix (`flag = UN`) are recorded by a lightweight, idempotent detector that runs on every ingestion, scanning the cumulative `tankers` dimension rather than only the current batch; the `NOT EXISTS` guard means each run inserts only flags not already present. Counts for both are reported in Table 7 and anomaly_flag count from tanker_positions is 411296.'

A third, more ambitious signal was implemented but proved to lie beyond the practical reach of the deployment hardware. The MMSI-swap detector (Listing 6) identifies a single IMO observed broadcasting more than one distinct MMSI the textbook footprint of identity manipulation [2] by aggregating across the entire staging history with `GROUP BY`. Executed over the full multi-billion-row corpus on physical-disk storage, this whole-history aggregation ran for approximately *19 hours* before it was terminated without completing.

```sql
SELECT TRIM(imo) AS imo, COUNT(DISTINCT TRIM(mmsi)) AS mmsi_count
FROM tanker_staging
WHERE LOWER(TRIM(ship_type)) = 'tanker'
  AND TRIM(imo) ~ '^[0-9]{7}$'
GROUP BY TRIM(imo)
HAVING COUNT(DISTINCT TRIM(mmsi)) > 1;
```

Listing 6: Whole-history MMSI-swap detection. The unbounded aggregation in the subquery is the source of its cost at corpus scale.

This non-result is reported deliberately, because it is informative. It establishes empirically that global aggregations over the full archive are infeasible under the project's hardware budget, and it is precisely this finding that motivates the day-parameterised, idempotent, separately-scheduled detection design analysed in Section 9.2. A consequence for interpretation, made explicit in Section 9.3, is that the dedicated anomaly tables reflect only *forward-going* detection on batches ingested after the detectors were deployed; historical anomalies are not back-filled, so every anomaly count in this chapter is a **lower bound**.

9 Discussion

9.1 Architecture and the principles it embodies

The platform's structure is a series of deliberate applications of established software and data-engineering principles rather than ad-hoc construction.

Staging-then-transform (ELT). Raw AIS is loaded unmodified into `tanker_staging` and only afterwards cleaned and projected into the `tankers` and `tanker_positions` tables. Because the transformation executes inside the database rather than in the loader, the pattern is more precisely *extract–load–transform* (ELT). Preserving the raw layer follows the staging-area discipline of dimensional warehouse design [20] and the broader principle of keeping an immutable system of record from which derived state can always be recomputed [15].

Decoupled, idempotent detection. Anomaly detection is separated from ingestion and written to be *idempotent*: each detector guards its inserts with `NOT EXISTS` predicates and partial unique indexes, so re-execution neither duplicates nor corrupts results [15]. This decoupling is what allows new detectors to be added and run *retroactively* over already-ingested data, and it is the direct architectural response to the infeasibility result of Section 8.4.

Priority-queued, demand-aware scheduling. Rather than relying solely on the scheduler's native cron triggers, ingestion is mediated by an application-level work queue (`data_consumer_queue`, Table 8). Jobs carry a priority tier daily ingestion outranks user-requested dates, which in turn outrank bulk backfill so that interactive demand can pre-empt long-running backfill. A consumer claims the highest-priority pending job under `FOR UPDATE SKIP LOCKED` (Listing 7), a pessimistic-lock idiom that guarantees exactly-once claiming without blocking concurrent consumers. A look-up against the archive (the "Package Table" in Table 9) makes the claim step *check-before-download*, so no date is fetched twice.

Table 8: The `data_consumer_queue` work queue (sample of 116 rows). Each job carries a `priority` tier and a `requester`: daily ingestion (100) pre-empts user-requested dates, which in turn outrank bulk backfill (tiered 20/15/10 here). The `ais_worker` claims the highest-priority `pending` job under `FOR UPDATE SKIP LOCKED` (Listing 7), marks it `in_progress`, and sets it `done` on success; the `check_already_done` task consults the archive (Table 9) before fetching, so no date is downloaded twice.

Queue id	Batch date	Priority	Requester	Status
1209	2026-05-27	100	daily	done
1206	2026-05-24	100	daily	in_progress
849	2026-04-22	20	backfill	done
814	2026-03-18	20	backfill	in_progress
790	2026-02-22	15	backfill	done
		...		
738	2026-01-01	10	backfill	done

```
cur.execute("""
    SELECT queue_id, source_batch_date
    FROM data_consumer_queue
    WHERE status = 'pending'
    ORDER BY priority DESC, created_at ASC
    LIMIT 1
    FOR UPDATE SKIP LOCKED
""")
```

Listing 7: Concurrency-safe job claiming: highest priority first, exactly-once under SKIP LOCKED.

Scheduled orchestration. The operational rhythm (Figure 5) is a *daily download* Scheduled orchestration. The operational rhythm is anchored by Airflow rather than raw cron, and the difference is substantive. A cron entry fires a command at a fixed instant with no notion of whether the triggered work can proceed: no retries, no task-level visibility, and crucially no way to wait on an external precondition. Ingestion here has exactly such a precondition. The Danish Maritime Authority publishes each day's archive retrospectively and at no guaranteed time. This means a fixed-time trigger would routinely

fire against a file that does not yet exist. The daily_package_sensor DAG separates when we begin looking from when the file appears: a cron-style schedule (0 15 * * *) starts the run, after which a PythonSensor in poke mode issues an HTTP HEAD probe every five minutes for up to nine hours and enqueues a single high-priority job. The sensor does no downloading itself, keeping the trigger lightweight. This conditional, observable, retry-aware scheduling is precisely what cron cannot express [21].

Table 9: The `data_date_archive` table (sample of 112 rows). It is the pipeline's checklist of dates already processed: the `check_already_done` task consults it before download, so a source package is never ingested twice and the database is not polluted with duplicate batches. The per-batch counts recorded here is how we calculated the ingestion funnel (Table 5) and the `/Stats` totals without scanning the fact table (*Tanker_Staging*).

Batch date	Staged rows	Tanker rows	Positions ins.	Archived at
2026-01-01	14516337	1302685	694339	2026-05-10 18:45
2026-01-02	14282506	1194115	665048	2026-05-10 17:08
2026-03-01	16455881	1644852	849833	2026-05-24 04:59
2026-04-22	30156066	3312205	1088904	2026-05-23 10:36
		...		
2026-05-26	23579760	2102924	925608	2026-05-29 20:00
2026-05-27	20206760	1733102	850880	2026-05-30 19:31

Layered architecture, DTO boundary, and RESTful exposure. The backend is partitioned into a data-access layer, a web/service layer, and the React frontend, with Data Transfer Objects decoupling the internal persistence entities from the public JSON contract a textbook application of the DTO pattern and separation of concerns [18]. The HTTP surface follows REST: resource-oriented, stateless endpoints, JSON representations, and explicit URI versioning (`/v1`, `/v2`) so that the contract can evolve without breaking existing clients [19].

9.2 Scaling constraints and engineering trade-offs

Several design decisions are best understood as conscious trade-offs forced by the corpus size and the hardware budget, and are worth defending explicitly.

Abandoning exact pagination totals. A conventional paged endpoint returns the total row count so clients can render page controls. On a fact table of this size, however, `COUNT(*)` over a filtered predicate is an $O(n)$ scan that dominated request latency. This means the linear processing time would go up, as database volume groups, this is undesired. The endpoint therefore returns `TotalItems = -1` as an explicit "not computed" marker and relies on offset paging without a precomputed total, trading a UI affordance for faster response and lower query latency. In a smaller database `COUNT(*)` would not be

an issue, but we found that the largest contribution to latency was counting total table size on `positions_table` and `staging_table`, therefore we prioritized responsiveness.

Dropping into raw SQL where the ORM cannot follow. The map is backed by a "latest position per active vessel" query (Listing 8), a pattern the LINQ provider cannot translate efficiently: PostgreSQL's `DISTINCT ON`, the canonical idiom for "latest row per group," has no LINQ equivalent. We therefore hand-wrote the query in raw SQL as a `LATERAL` correlated look-up — for each active vessel, an inner subquery takes the single most recent position (`ORDER BY timestamp_utc DESC LIMIT 1`) projected onto a keyless entity type (`HasNoKey()`). Backed by the composite `(tanker_id, timestamp_utc DESC)` index, each look-up is an index scan that stops at the first row, so the query's cost scales with the number of *vessels* (a few thousand) rather than the number of *positions* (tens of millions). The portable `GROUP BY` formulation the ORM could produce was markedly slower precisely because it aggregates over the full fact table instead of probing the index. Accepting this controlled departure from the ORM abstraction. This creates a mismatch between the Map endpoint and the other endpoints [18] but it was the better trade-off, because we prioritized user-friendliness over consistency cross the API controllers/DataService.

```
FROM tankers t
CROSS JOIN LATERAL (
    SELECT p.latitude, p.longitude, p.timestamp_utc, p.sog, p.cog, p.
       heading
    FROM tanker_positions p
    WHERE p.tanker_id = t.tanker_id
    ORDER BY p.timestamp_utc DESC
    LIMIT 1
) p
WHERE t.is_active = TRUE;
```

Listing 8: Latest position per active tanker: a per-vessel correlated LATERAL look-up that the ORM could not express.

Selective indexing matched to query patterns. Rather than indexing every column, we added indexes only where the system's actual queries need them. The composite `(tanker_id, timestamp_utc DESC)` index is built for the latest-position look-up: because the rows are already ordered by time within each vessel, the query can read the single newest position per vessel and stop, instead of scanning all of that vessel's history. The partial indexes cover only the rows that matter. This makes sure our database stay as small and cheaply to maintain as possible. This restraint matters on a write-heavy table of this size: every extra index has to be updated on every insert, so it slows down ingestion and takes up storage. We deliberately avoided adding indexes everywhere.

Heavy detection as a separate DAG. The whole-history MMSI-swap scan (Section 8.4) ran for roughly 19 hours without completing and was abandoned. Its cost

comes from aggregating across the entire staging history in a single pass, which is infeasible on the deployment hardware. As an immediate measure we isolated the scan in its own weekly DAG so that, however long it runs, it never delays daily ingestion. This addresses *when* the scan runs, but not its underlying cost: the scan still attempts the full corpus in one go. The remaining step, left for future work, is to bound *how much* it scans, running it over date-limited slices rather than the whole history, so that each execution completes within a predictable time. The failed whole-corpus run is thus the empirical justification for both decisions: it is why heavy detection is scheduled separately, and why a bounded, incremental formulation is the right target for making it complete.

9.3 Limitations and threats to validity

Anomaly coverage is uneven across detectors. The two detectors differ sharply in coverage. The invalid-MID check runs over the full `tankers` dimension, so it covers every vessel in the corpus regardless of when it was ingested. It completed successfully but returned *zero* flags: all 1925 distinct tankers resolved to a valid ITU country code, with none falling back to the `UN`. This is a clean negative result. I means across the processed corpus, no tanker carried an unresolved MID prefix, but not a failed detector. The MMSI-swap signal is different: the whole-history scan was abandoned after running for roughly 19 hours without completing (Section 8.4), and no date-bounded (per-batch) alternate version yet runs, so swap detection is effectively absent over the historical archive.

9.4 Reflection on the emergent contribution

The most eye-catching possible outcome positively confirming illicit vessel behavior lies beyond this project's evidentiary reach, and the limitations above explain why. Another approach would have been a machine-learning process with Ground-Truth data. One vision we had was that platform could serve as portal for users to engage in human-in-the-loop labeling themselves, and produce such a clean dataset for future Neural Networks. This didn't emerge.

What the system *does* demonstrably achieve is a reproducible, automated, and horizontally-replayable pipeline that ingests a multi-billion-row AIS archive, reduces and validates it into queryable analytical tables, and *surfaces* identity- and flag-irregularity signals at that scale while rendering them inspectable through an API. The contribution is therefore methodological and infrastructural rather than investigative and, judged on the engineering criteria appropriate to such a system (correctness at scale, principled architecture, and honest characterization of its own limits), it stands on its own terms.

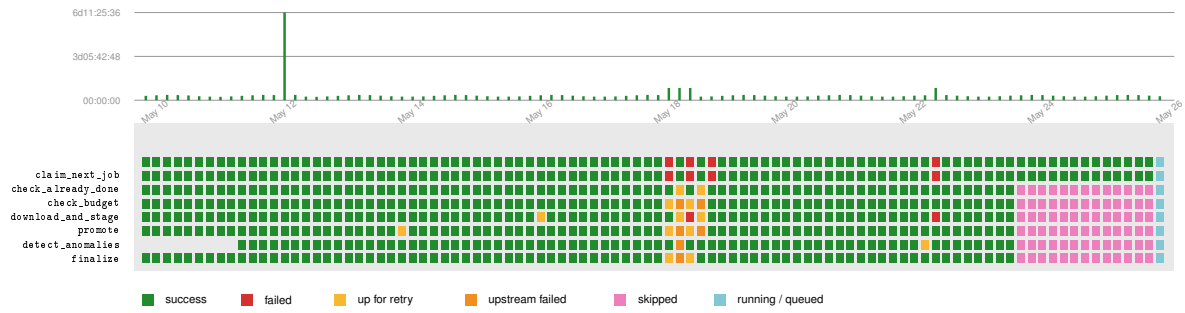


Figure 10: Grid view of the consumer worker DAG. Successes dominate; the cluster around May 21 shows a retry/upstream-failure incident, this is just application down-time, and the pink block reflects runs where `claim_next_job` found no work and short-circuited the downstream tasks (as it should).

10 Conclusion

This project set out to address a problem that is, at root, infrastructural. Tanker AIS data is abundant, yet raw national archives are too voluminous, heterogeneous, and error-laden to be usable without dedicated tooling, and the identity and flag irregularities that matter most for tanker monitoring are buried within that volume. The work delivers a complete, automated, end-to-end platform that ingests the historical AIS record of tanker vessels, reduces and validates it into queryable analytical tables, surfaces identity and flag irregularity signals at archive scale, and renders both the cleaned data and those signals inspectable through a versioned REST API and a client-side website. Every stage runs on our own physical hardware within the EU, free of any dependence on commercial cloud. The central engineering achievement is a pipeline that has ingested and processed over two billion raw position reports; 2,044,611,833 staged rows across the 2026 archive and its ongoing historical backfill, while remaining conscious of both memory and storage. Built on an extract-load-transform design, the pipeline preserves every batch in `tanker_staging` as an immutable system of record before any interpretation occurs; chunked bulk `COPY` ingestion holds the working set independent of file size; and a sustained mean rate of roughly 5,585 rows per second carries even the largest single daily archive of over 30 million rows in about 69 minutes. Every number in this funnel: all the staged mobiles, the ones classified as tankers, and the 92.4 million positions written to the fact table, was counted as each batch came in, not by querying the fact table afterward. The system can therefore describe itself at scale without ever paying the cost of scale. Ingestion was made robust and demand-aware through three cooperating Airflow DAGs and an application-level priority queue. A lightweight sensor polls the source with HTTP `HEAD` probes and enqueues a job only once the day's archive genuinely appears; a single-writer worker claims the highest-priority pending job under `FOR UPDATE SKIP LOCKED` and drives a linear seven-task pipeline; and the most expensive detector is isolated to its own weekly window. Because the worker is independent, date-parameterised, and replayable, routine daily ingestion and bulk historical backfill coexist without resource contention. This is a property we actively exploit as the archive continues to be filled backward in time. Within this pipeline, identity and flag irregularity detection was

implemented as a set of idempotent, decoupled SQL detectors, ranging from a per-batch invalid-MID check cheap enough to run on every ingestion, to a whole-history MMSI-swap scan across the full corpus. The latter did not complete within roughly 19 hours and was terminated, and we report this deliberately as an empirical result: it establishes the computational ceiling of naive whole-archive aggregation on the deployment hardware, and it is precisely this finding that justifies the day-parameterised, separately-scheduled detection architecture. A constraint that defeated the naive approach thereby became the evidence for the design moving forward. The processed data and the surfaced signals were exposed through a cleanly layered ASP.NET Core service, with a Data Transfer Object boundary insulating the public JSON contract from the persistence schema, and explicit URI versioning so the contract can evolve without breaking existing clients. We built a React and TypeScript single-page application that provides an interactive documentation page, that serves as a working client of the public API; issuing live requests and rendering real responses. This leaves the system not both accessible and self-demonstrating.

References

[1] International Maritime Organization, *AIS transponders*, https://www.imo.org/en/OurWork/Safety/Pages/AIS.aspx, IMO description of AIS carriage requirements and purpose. Add the access date you used.

[2] G. Pallotta, M. Vespe, and K. Bryan, "Vessel pattern knowledge discovery from AIS data: A framework for anomaly detection and route prediction," *Entropy*, vol. 15, no. 6, pp. 2218–2245, 2013, The TREAD framework. DOI: 10.3390/e15062218.

[3] A. Androjna, I. Pavić, L. Gucma, P. Vidmar, and M. Perkovič, "AIS data manipulation in the illicit global oil trade," *Journal of Marine Science and Engineering*, vol. 12, no. 1, p. 6, 2024. DOI: 10.3390/jmse12010006.

[4] T. Emmens, C. Amrit, A. Abdi, and M. Ghosh, "The promises and perils of Automatic Identification System data," *Expert Systems with Applications*, vol. 178, p. 114 975, 2021. DOI: 10.1016/j.eswa.2021.114975.

[5] P. Last, M. Hering-Bertram, and L. Linsen, "A real-time ais data cleaning algorithm," *TransNav: International Journal on Marine Navigation and Safety of Sea Transportation*, vol. 9, no. 3, pp. 309–314, 2015. DOI: 10.12716/1001.09.03.04.

[6] Y. Zhao, "A new classification method for ship trajectories based on AIS data," *Journal of Marine Science and Engineering*, vol. 11, no. 9, p. 1646, 2023. DOI: 10.3390/jmse11091646.

[7] K. Duan et al., "AIS-based operational phase identification using Progressive Ablation Feature Selection with machine learning for improving ship emission estimates," *Journal of the Air & Waste Management Association*, vol. 74, no. 2, pp. 100–115, 2024. DOI: 10.1080/10962247.2023.2274348.

[8] N. Newaliya, V. Siwach, H. Sehrawat, and Y. Singh, "Exploring maritime movement information: An explainable AI approach using Hi-DBSCAN and SHAP," *International Journal of Systematic Innovation*, vol. 8, no. 4, pp. 131–145, 2024. DOI: 10.6977/IJoSI.202412_8(4).0009.

[9] D. Vergara, X. Lang, M. Zhang, M. Alexandersson, and W. Mao, "Reduced environmental impact of short sea shipping through optimal propulsion power allocation," *Journal of Cleaner Production*, 2025, Two-stage propulsion power allocation framework using MS-PELT segmentation and PCDP optimisation. Please verify volume/-pages from the published version once available. DOI: 10.1016/j.jclepro.2025.145951.

[10] T. Uyanık et al., "Data-driven approach for estimating power and fuel consumption of ship: A case of container vessel," *Mathematics*, vol. 10, no. 22, p. 4167, 2022, Compares ANN, DNN, Bayesian regression, XGBoost – matches the methods you describe. DOI: 10.3390/math10224167.

[11] M. Zhang, N. Tsoulakos, P. Kujala, and S. Hirdaris, "A deep learning method for the prediction of ship fuel consumption in real operational conditions," *Engineering Applications of Artificial Intelligence*, vol. 130, p. 107 425, 2024. DOI: 10.1016/j.engappai.2023.107425.

[12] H. N. Psaraftis and C. A. Kontovas, "Speed models for energy-efficient maritime transportation: A taxonomy and survey," *Transportation Research Part C*, vol. 26, pp. 331–351, 2013. DOI: 10.1016/j.trc.2012.09.012.

[13] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Computing Surveys*, vol. 51, no. 5, p. 93, 2018. DOI: 10.1145/3236009.

[14] Y. Yang, Y. Liu, G. Li, Z. Zhang, and Y. Liu, "Harnessing the power of machine learning for AIS data-driven maritime research: A comprehensive review," *Transportation Research Part E: Logistics and Transportation Review*, vol. 183, p. 103 426, 2024. DOI: 10.1016/j.tre.2024.103426.

[15] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[16] Danish Maritime Authority, *AIS data*, http://aisdata.ais.dk/, Primary data source for this project. Add the access date you used.

[17] K. Schwaber and J. Sutherland. "The Scrum guide: The definitive guide to Scrum: The rules of the game," Accessed: May 31, 2026. [Online]. Available: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf.

[18] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[19] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[20] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Wiley, 2013.

[21] Apache Software Foundation, *Apache Airflow documentation*, https://airflow.apache.org/docs/, Accessed: 2026-05-31, 2024.